

CS 4/53401 – Secure Programming BIBIFI Project

Build It; Break It; Fix It (BIBIFI)

Secure Art Gallery Web Application

Weeks 4–11 | September 14–November 6, 2026

Teams of 2 | 300 Points

Project Goal

You will design, build, attack, and secure a web-based application for managing an art gallery. The system tracks guests and employees entering and leaving the gallery and individual rooms.

Alignment with the Original UMD BIBIFI Contest

This project follows the design of the original University of Maryland Build It, Break It, Fix It (BIBIFI) secure-development contest (Ruef et al., CCS 2016; Parker et al., TOPS 2020). In the original contest, build-it teams are scored on the correctness, performance, and security of their submissions; break-it teams earn points by submitting concrete, reproducible test cases that expose defects in another team's system, with security-relevant vulnerabilities weighted four times more heavily than ordinary correctness bugs; and scoring between builders and breakers is zero-sum, so a validated vulnerability both penalizes the build-it team and rewards the break-it team that found it. Duplicate discoveries of the same underlying flaw split their points evenly across the teams that reported it, which rewards genuinely novel findings over re-treading known ground. This project adapts that structure to a single-semester, two-student-team classroom format; see Section 2.2 for the specific scoring mechanics.

The project follows the BIBIFI model:

- Build It: Design and implement a functional and secure application.
- Break It: Attack other teams' applications (via an anonymized code exchange) and identify vulnerabilities using reproducible test cases.
- Fix It: Correct vulnerabilities and demonstrate that the fixes work.

The goal is to experience software security from three perspectives: developer, attacker, and security engineer.

1. Required System

Your project must be a functioning web application backed by a database.

1.1 Required Architecture

Browser → Web Application → Backend/API → Database

The system must include:

- Web-based user interface
- Backend application or REST-style API
- Relational database
- Authentication
- Authorization/access control
- Gallery event management

- Audit/security logging

Docker/Docker Compose is strongly recommended.

1.2 Users and Roles

At minimum, implement:

- User/Guest: Login/logout and access authorized information.
- Employee: Record authorized gallery events and view permitted information.
- Administrator: Manage users/roles and authorized administrative information.

Authorization must be enforced on the server side.

1.3 Gallery Functionality

The application must support:

- Person enters the gallery.
- Person leaves the gallery.
- Person enters a room.
- Person leaves a room.
- View current gallery/room occupancy.
- Search and query historical events.
- Administrative functions.

The original BIBIFI concepts are retained: logappend → Record gallery events; logread → Query/reconstruct gallery state. These concepts may be implemented as secure backend/API operations rather than separate command-line programs.

2. Database and Security Requirements

A database is mandatory. Persistent data may not be stored only in text files, JSON files, or memory.

Suggested entities include: Users, Persons, Rooms, GalleryEvents, Sessions, AuditLogs

Your implementation must address:

- Secure password storage
- Authentication
- Authorization
- Input validation
- Output encoding
- SQL injection prevention
- Cross-Site Scripting (XSS)
- Broken access control / IDOR
- Session/token security
- Information disclosure
- Secure error handling
- Secrets management
- Audit logging

- Security misconfiguration

Passwords must never be stored in plaintext. Database queries must use parameterized/prepared statements or an equivalent safe mechanism. Secrets such as database passwords and API keys must not be committed to GitHub.

2.1 Business-Logic Security

The backend must prevent invalid state transitions. Examples:

- A person cannot leave the gallery if they are not inside.
- A person cannot leave a room they do not occupy.
- A person cannot occupy multiple rooms simultaneously.
- Duplicate/out-of-order events cannot create an invalid state.
- Unauthorized users cannot modify protected records.

2.2 BIBIFI Scoring Model

The following mechanics, drawn directly from the original UMD BIBIFI contest, govern how points move between phases and teams:

- Correctness oracle: the instructor supplies an automated suite of correctness/functional tests that is run against every Build It submission, mirroring BIBIFI's automated correctness grading.
- Security-weighted breaking: a valid, reproducible security vulnerability is worth 4× the points of an ordinary correctness/functional bug.
- Zero-sum scoring: each confirmed vulnerability both deducts points from the build-it team's Build It score and adds points to the break-it team's Break It score.
- Duplicate breaks: when more than one break-it team reports the same underlying vulnerability, its points are split evenly among the teams that reported it.
- Evidence requirement: a valid break must include an executable test case — a concrete request/input plus the observed vs. expected behavior — reproducible against the target system. A narrative description alone is not sufficient.
- Fix decay: after Break It results are released, any confirmed vulnerability left unpatched costs the build-it team 1 additional point per day (per vulnerability) until it is fixed or the Fix It deadline passes, whichever comes first.

3. Secure Design Requirement

Security must be considered before implementation. Each team must produce:

- Architecture Diagram
- Data-Flow Diagram (DFD)
- Database/ER Diagram
- STRIDE Threat Model
- Security Requirements

The DFD must identify:

- External entities
- Processes
- Data stores

- Data flows
- Trust boundaries

3.1 STRIDE

Threat	Question
Spoofing	Can an attacker impersonate a user?
Tampering	Can data be modified without authorization?
Repudiation	Can actions be denied or hidden?
Information Disclosure	Can sensitive information be exposed?
Denial of Service	Can the application be made unavailable?
Elevation of Privilege	Can a user gain unauthorized permissions?

Identify at least six specific threats. For each threat, describe the component, attack scenario, impact, and mitigation.

4. Week-by-Week Schedule

Week	Dates	Phase	Activities / Deliverable
4	Sep. 14–18	Setup	Teams, GitHub, technology stack, users/roles, functional and security requirements. Deliverable: Team + Requirements
5	Sep. 21–25	Secure Design	Architecture, DFD, trust boundaries, ERD, STRIDE, security requirements. Deliverable: Design Document
6	Sep. 28–Oct. 2	Build It I	Database, backend/API, authentication, authorization, core functionality. Milestone: Backend + Database
7	Oct. 5–9	Build It II	Web interface, gallery events, state validation, audit logging. Milestone: Functional Web App
8	Oct. 12–16	Build It III	Security controls, SQLi/XSS defenses, session security, security testing. Milestone: Security-Complete Build
9	Oct. 19–23	Build It Final	Testing, documentation, deployment, security review, correctness oracle run. Deliverable: Build It
10	Oct. 26–30	Break It	Anonymized code exchange with an assigned opposing team; attack using provided test harness; submit valid breaks as executable test cases (security bugs weighted 4×). Deliverable: Break It Reports
11	Nov. 2–6	Fix It	Fix vulnerabilities, regression testing, final documentation and demonstration. Deliverable: Fix It + Final Demo

5. Weekly Expectations

Week 4 – Project Setup and Requirements

- Form teams of two.
- Create the GitHub repository.
- Select the technology stack.

- Define users and roles.
- Define functional requirements.
- Define initial security requirements.
- Set up the development environment.

Deliverable: Team information, GitHub repository, application description, functional requirements, and initial security requirements.

Week 5 – Secure Design and Threat Modeling

Create the architecture, DFD, ERD/database design, trust boundaries, STRIDE threat model, and explicit security requirements. Deliverable: Secure Design Document.

Week 6 – Build It I

Implement:

- Database schema and initialization
- Backend/API
- User accounts
- Password hashing
- Login/logout
- Sessions/tokens
- Roles and authorization

Milestone: Demonstrate Browser → Backend → Database with working authentication.

Week 7 – Build It II

Implement the dashboard, gallery/room status, persons, event recording, event history, queries, administrative functions, and server-side state validation. Milestone: Functional Web Application.

Week 8 – Build It III: Security Engineering

Implement and test defenses against:

- SQL Injection
- XSS
- IDOR/broken access control
- Authentication attacks
- Privilege escalation
- Session/token attacks
- Input-validation bypass
- Information disclosure
- Security misconfiguration

Recommended tools include OWASP ZAP, Burp Suite, curl, Postman, Nmap, Semgrep, and SQLMap where permitted.

Week 9 – Build It Submission

Complete functional and security testing. Verify that another student can install and run the application. The instructor's automated correctness test suite (oracle tests) will be run against your submission as part of Build It scoring — see Section 2.2.

Submit:

- GitHub repository
- README
- Source code
- Database schema
- Tests
- Design document
- Threat model
- API documentation
- Deployment instructions

Week 10 – Break It

Each build-it team's repository is exchanged, anonymized, with one assigned opposing team, matching the head-to-head structure of the original contest. Break-it teams attack their assigned application and identify vulnerabilities.

Potential targets include SQL injection, XSS, IDOR, broken access control, authentication bypass, privilege escalation, session attacks, CSRF, information disclosure, business-logic flaws, race conditions, and security misconfiguration.

Each vulnerability report must contain:

- Vulnerability title
- Affected component/endpoint
- Severity
- An executable test case demonstrating the attack (required — see Section 2.2)
- Attack description
- Steps to reproduce
- Evidence
- Security impact
- Root cause, if identifiable
- Recommended mitigation

Reminder: confirmed security vulnerabilities are worth 4× the points of correctness/functional bugs, scoring is zero-sum against the build-it team, and duplicate reports of the same flaw split points evenly across the reporting teams.

Week 11 – Fix It and Final Demonstration

For significant vulnerabilities:

- Reproduce the vulnerability.
- Identify the root cause.
- Identify the violated security requirement.

- Implement the fix.
- Add a regression test.
- Verify that the vulnerability is fixed.
- Update the documentation.

Recall the fix-decay rule from Section 2.2: any confirmed vulnerability that remains unpatched continues costing the build-it team points for every day it stays open, up to the Fix It deadline.

The final demonstration must show the working application, database, authentication, authorization, gallery functionality, secure design, STRIDE model, security controls, an attack, and the corresponding fix.

6. Required Repository Structure

project-root/

- src/ — backend/, frontend/
- database/ — schema.sql, seed.sql
- tests/ — functional/, security/
- docs/ — design_document.pdf
- docker-compose.yml
- README.md
- .gitignore

The README must explain the project purpose, team members, technology stack, architecture, installation, database setup, roles, API endpoints, testing, security controls, and known limitations.

7. 300-Point Grading Rubric

7.1 Build It – 100 Points

Category	Points
Web Application	10
Backend/API	10
Database Design & Implementation	10
Authentication	10
Authorization/Access Control	10
Gallery Business Logic	10
Secure Coding & Input Validation	10
Automated Correctness (Oracle Tests)	10
Secure Design & Documentation	10
Deployment/Reproducibility	10
TOTAL	100

7.2 Break It – 100 Points

Category	Points
Security vulnerabilities (weighted 4×, test case required)	50
Correctness/functional bugs (weighted 1×, test case required)	15
Severity and security impact analysis	15

Technical/root-cause analysis	10
Quality & reproducibility of vulnerability reports	10
TOTAL	100

Points for a given vulnerability are zero-sum with the build-it team's score, and are split evenly if more than one break-it team reports the same underlying flaw (Section 2.2).

7.3 Fix It – 50 Points

Category	Points
Correct vulnerability fixes	20
Root-cause analysis	10
Quality of mitigation	10
Regression/security testing	5
Documentation updates	5
TOTAL	50

Confirmed vulnerabilities left unpatched after Break It results are released lose the build-it team 1 point per day (per vulnerability) until fixed or the Fix It deadline, whichever comes first.

7.4 Final Demonstration – 50 Points

Category	Points
Application functionality	10
Web application + database	5
Authentication + authorization	5
Secure design + threat model	5
Security controls	10
Attack demonstration	5
Fix demonstration	5
Technical understanding/team presentation	5
TOTAL	50

8. Final Point Distribution

Phase	Points
Build It	100
Break It	100
Fix It	50
Final Demonstration	50
TOTAL	300

Project Philosophy

Requirements → Design → Build → Test → Attack → Fix → Verify

During Build It, think like a developer. During Break It, think like an attacker. During Fix It, think like a security engineer.

As in the original UMD contest, the zero-sum, security-weighted scoring model is deliberate: it rewards teams for building software that resists real scrutiny, not just software that runs. The goal is not only to

build software that works, but to demonstrate that you can design, build, test, attack, and improve secure software.